

```
In [1]: import pandas as pd                    #Step 1: Import Required Libraries
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [16]: df = pd.read_csv('C:/Users/hp/Desktop/air_quality_india_sample.csv')    #Step 2:
df.head()
```

```
Out[16]:
```

	Date	City	PM2.5	PM10	NO2	SO2	CO	O3
0	2020-01-01	Bengaluru	42	72	18	9	0.6	25
1	2020-01-02	Bengaluru	39	65	16	8	0.5	27
2	2020-01-03	Bengaluru	45	70	20	10	0.7	24
3	2020-01-04	Bengaluru	50	80	22	11	0.8	26
4	2020-01-05	Bengaluru	47	78	19	10	0.6	28

```
In [3]: df.info()                            #Step 3: Basic Info & Missing Value Check
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Date    5 non-null      object
1   City    5 non-null      object
2   PM2.5   5 non-null      int64
3   PM10    5 non-null      int64
4   NO2     5 non-null      int64
5   SO2     5 non-null      int64
6   CO      5 non-null      float64
7   O3      5 non-null      int64
dtypes: float64(1), int64(5), object(2)
memory usage: 452.0+ bytes
```

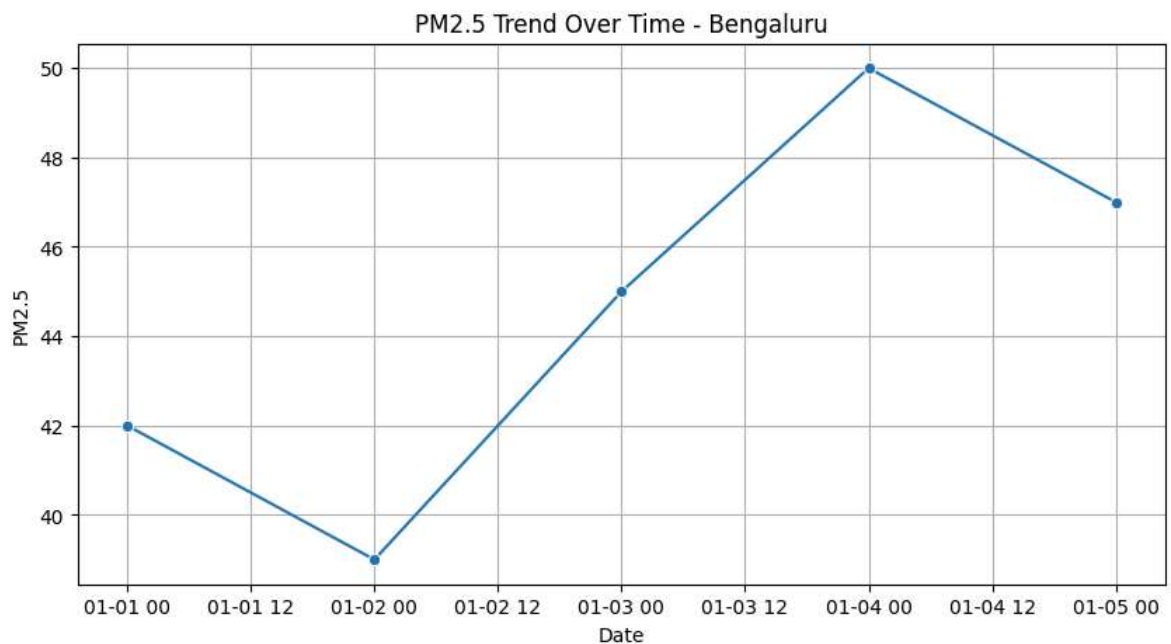
```
Out[3]: Date      0
City      0
PM2.5     0
PM10      0
NO2       0
SO2       0
CO        0
O3        0
dtype: int64
```

```
In [4]: df = df.dropna()    # or fillna() if needed
```

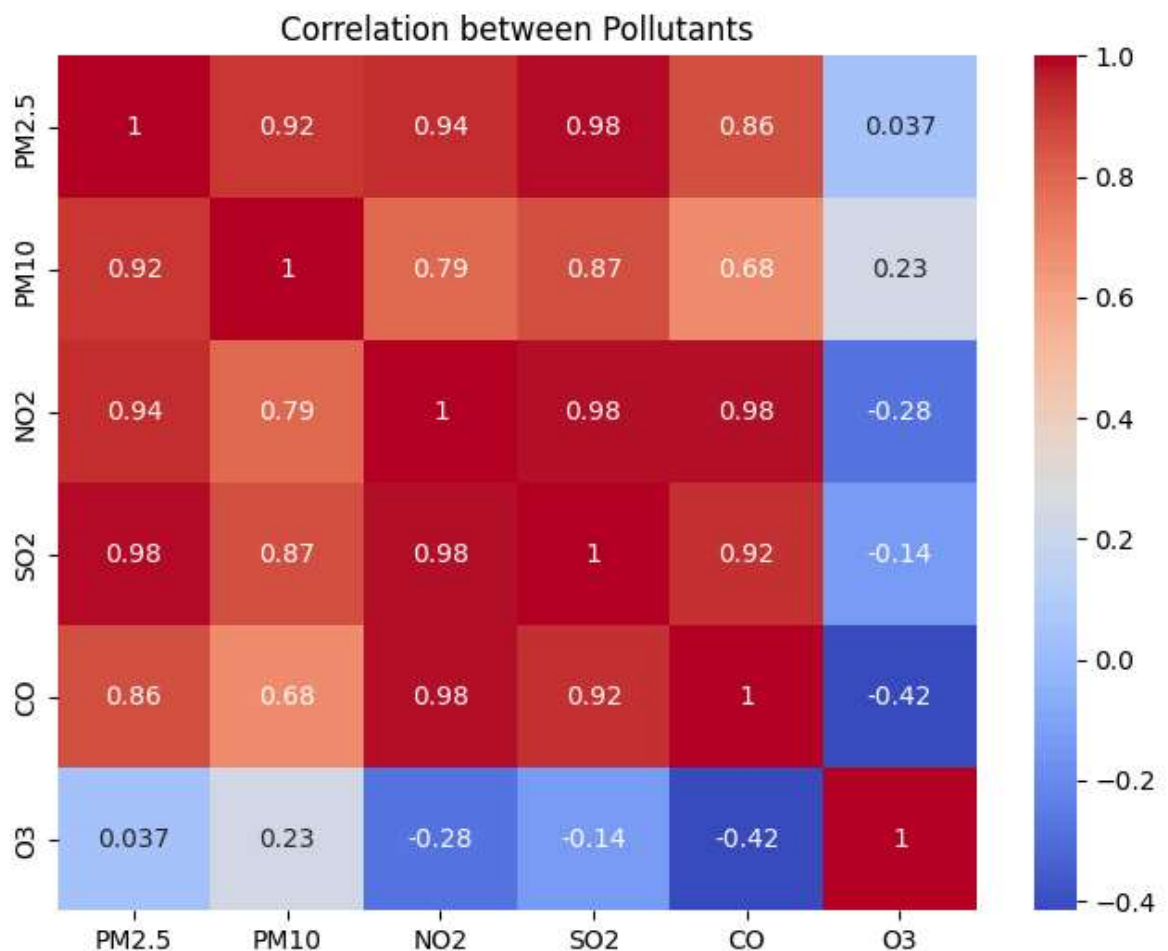
```
In [6]: df['Date'] = pd.to_datetime(df['Date'])    #Step 4: Convert Date Column to Dat
```

```
In [7]: plt.figure(figsize=(10, 5))                # Step 5: Visualize PM2.5 Trend Ov
sns.lineplot(data=df, x="Date", y="PM2.5", marker='o')
plt.title("PM2.5 Trend Over Time - Bengaluru")
```

```
plt.ylabel("PM2.5")
plt.xlabel("Date")
plt.grid()
plt.show()
```



```
In [8]: plt.figure(figsize=(8, 6)) # Step 6: Correlation Heatmap
sns.heatmap(df[['PM2.5', 'PM10', 'NO2', 'SO2', 'CO', 'O3']].corr(), annot=True,
plt.title("Correlation between Pollutants")
plt.show()
```



```
In [10]: #Step 7: Prepare Data for Prediction (ML)
X = df[['PM10', 'NO2', 'SO2', 'CO', 'O3']] # Features
y = df['PM2.5'] # Target
```

```
In [12]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

```
In [13]: model = LinearRegression() # Step 9: Train Linear Regression Model
model.fit(X_train, y_train)
```

```
Out[13]: LinearRegression
```

```
LinearRegression()
```

```
In [14]: y_pred = model.predict(X_test) #Step 10: Predict and Evaluate
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R² Score:", r2_score(y_test, y_pred))
```

Mean Squared Error: 3.339097101455391

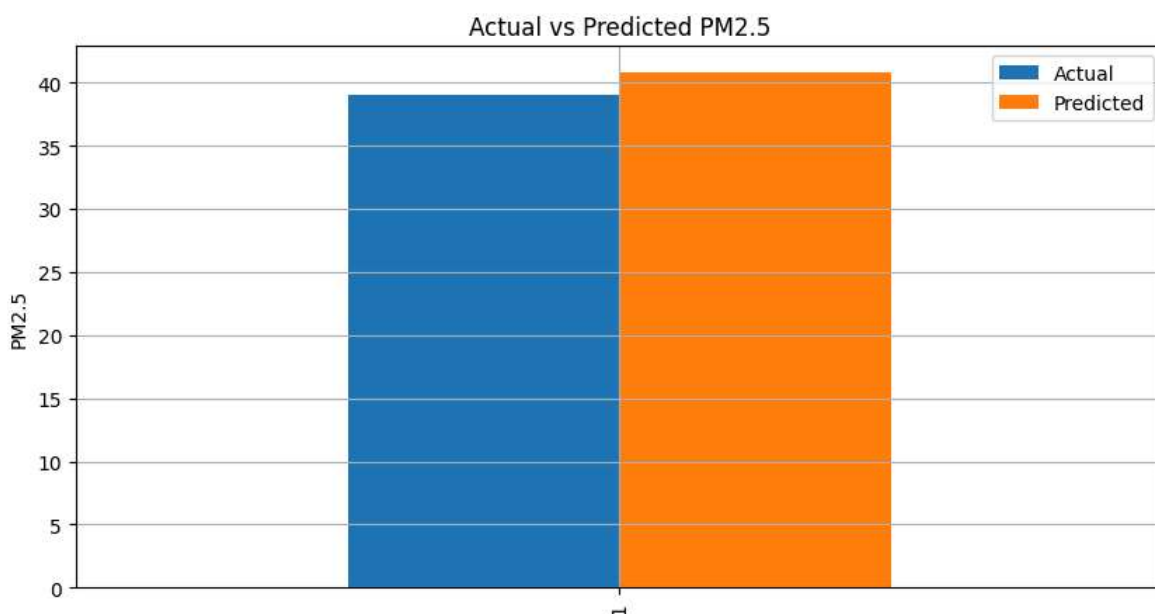
R² Score: nan

C:\Users\hp\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics_regression.py:1266: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.

warnings.warn(msg, UndefinedMetricWarning)

```
In [15]: df_compare = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred}) #Step 11: C
print(df_compare)
df_compare.plot(kind='bar', figsize=(10, 5))
plt.title("Actual vs Predicted PM2.5")
plt.ylabel("PM2.5")
plt.grid()
plt.show()
```

	Actual	Predicted
1	39	40.82732



```
In [ ]:
```